

Lógica Computacional 2024/2025 - TP3

Grupo 6

- Cláudia Faria, a105531
- Patrícia Bastos, a102502

```
In [ ]: !pip install pysmt
!pysmt-install --z3
from z3 import *
```

Exercício 3

Considere de novo o 1º problema do trabalho TP2 relativo à descrição da cifra A5/1 e o FOTS usando BitVec's que aí foi definido para a componente do gerador de chaves. Ignore a componente de geração final da chave e restrinja o modelo aos três LFSR's.

Sejam X_0, X_1, X_2 as variáveis que determinam os estados dos três LFSR's que ocorrem neste modelo. Como condição inicial e condição de erro use os predicados

$$I \equiv (X_0 > 0) \wedge (X_1 > 0) \wedge (X_2 > 0) \text{ e } E \equiv \neg I$$

a. Codifique em “z3” o SFOTS assim definido.

b. Use o algoritmo PDR “property directed reachability” (codifique-o ou use uma versão pré-existente) e, com ele, tente provar a segurança deste modelo.

Resolução

Vamos começar por codificar o SFOTS.

```
In [ ]: from z3 import *

def declare(i): # Declara as variáveis
    return {
        'LFSR_0': BitVec(f'LFSR_0_{i}', 19),
        'LFSR_1': BitVec(f'LFSR_1_{i}', 22),
        'LFSR_2': BitVec(f'LFSR_2_{i}', 23)
    }

def init(state): # Define os estados iniciais
    return And(
        UGT(state['LFSR_0'], BitVecVal(0, 19)),
        UGT(state['LFSR_1'], BitVecVal(0, 22)),
        UGT(state['LFSR_2'], BitVecVal(0, 23))
    )

def proximo_estado(s): # Calcula o próximo estado

    c0 = Extract(8, 8, s['LFSR_0'])
    c1 = Extract(10, 10, s['LFSR_1'])
    c2 = Extract(10, 10, s['LFSR_2'])

    maj = (c0 & c1) | (c1 & c2) | (c2 & c0)

    updated_LFSR_0 = If(c0 == maj,
        Concat(Extract(17, 0, s['LFSR_0']),
            Extract(18, 18, s['LFSR_0']) ^ Extract(17, 17, s['LFSR_0']) ^
            Extract(16, 16, s['LFSR_0']) ^ Extract(13, 13, s['LFSR_0'])),
        s['LFSR_0'])

    updated_LFSR_1 = If(c1 == maj,
        Concat(Extract(20, 0, s['LFSR_1']),
            Extract(21, 21, s['LFSR_1']) ^ Extract(20, 20, s['LFSR_1'])),
        s['LFSR_1'])

    updated_LFSR_2 = If(c2 == maj,
        Concat(Extract(21, 0, s['LFSR_2']),
            Extract(22, 22, s['LFSR_2']) ^ Extract(21, 21, s['LFSR_2']) ^
            Extract(20, 20, s['LFSR_2']) ^ Extract(7, 7, s['LFSR_2'])),
        s['LFSR_2'])

    return {
        'LFSR_0': updated_LFSR_0,
        'LFSR_1': updated_LFSR_1,
        'LFSR_2': updated_LFSR_2
    }

def trans(curr, prox): # Define relação de transição
    proximo = proximo_estado(curr)
    return And(
        prox['LFSR_0'] == proximo['LFSR_0'],
        prox['LFSR_1'] == proximo['LFSR_1'],
        prox['LFSR_2'] == proximo['LFSR_2']
    )

def error(state): # Verifica a propriedade de erro, ou seja, se algum valor é zero ou negativo.
    return Or(
        ULE(state['LFSR_0'], BitVecVal(0, 19)),
        ULE(state['LFSR_1'], BitVecVal(0, 22)),
        ULE(state['LFSR_2'], BitVecVal(0, 23))
    )

def bin_format(model, bv, width): # Formata um BitVec como string binária
    value = model.eval(bv).as_long()
    return format(value, f'0{width}b')
```

```
def genTrace(vars, init, trans, error, n): # Gera um trace de n estados usando vars para determinar dinamicamente as variáveis.
    solver = Solver()

    # Define os estados
    states = [declare(i) for i in range(n + 1)]

    # Aplica a condição inicial
    solver.add(init(states[0]))

    # Adiciona transições
    for i in range(n):
        solver.add(trans(states[i], states[i + 1]))

    # Adiciona condição de erro para cada estado
    for state in states:
        solver.add(Not(error(state)))

    # Verifica satisfatibilidade
    if solver.check() == sat:
        model = solver.model()
        for i in range(n + 1):
            print(f"\nEstado: {i}")
            for var in vars:
                print(f"    {var} = {bin_format(model, states[i][var], 19 if var == 'LFSR_0' else 22 if var == 'LFSR_1' else 23)}")
            return True
        else:
            print("Condição de erro alcançada.")
            return False

def test_sfots(n): # Testa a execução do sistema com n transições.
    print(f"Testar SFOTS com {n} transições:")
    vars = ['LFSR_0', 'LFSR_1', 'LFSR_2']
    return genTrace(vars, init, trans, error, n)

test_sfots(7)
```

Testar SFOTS com 7 transições:

```
Estado: 0
LFSR_0      = 000000000101111100
LFSR_1      = 0000000010010011110010
LFSR_2      = 000000000000000100000

Estado: 1
LFSR_0      = 0000000001011111000
LFSR_1      = 00000000100100111100100
LFSR_2      = 0000000000000000100000

Estado: 2
LFSR_0      = 00000000010111110000
LFSR_1      = 00000001001001111001000
LFSR_2      = 00000000000000001000000

Estado: 3
LFSR_0      = 00000000010111110000
LFSR_1      = 0000010010011110010000
LFSR_2      = 000000000000000010000000

Estado: 4
LFSR_0      = 00000000101111100000
LFSR_1      = 0000100100111100100000
LFSR_2      = 000000000000000010000000

Estado: 5
LFSR_0      = 00000001011111000000
LFSR_1      = 0001001001111001000000
LFSR_2      = 000000000000000010000000

Estado: 6
LFSR_0      = 0000010111110000000
LFSR_1      = 0010010011110010000000
LFSR_2      = 000000000000000010000000

Estado: 7
LFSR_0      = 0000101111100000001
LFSR_1      = 0100100111100100000000
LFSR_2      = 000000000000000010000000
```

Out[]: True

O algoritmo Property Directed Reachability (PDR), também conhecido como IC3 (Incremental Construction of Inductive Clauses for Indubitable Correctness), é uma técnica usada na verificação formal de sistemas. Ele determina se uma propriedade de segurança é válida num sistema transitivo (geralmente representado como um modelo de transição de estados) ou se existe um contraexemplo (caminho que viole a propriedade).

Usando o seguinte algoritmo PDR implementado por Peter Den Hartog, em <https://github.com/pddenhar/Z3-IC3-PDR/blob/master/pdr.py>.

```
In [ ]: # Implementation of the PDR algorithm by Peter Den Hartog. Apr 28, 2016

from z3 import *

# a tcube is a conjunction of literals associated with a given frame (t) in the trace
class tCube(object):
    #make a tcube object associated with frame t. If t is none, have it be frameless
    def __init__(self, model, lMap, t = None):
        self.t = t
        #filter out primed variables when creating cube
        self.cubeLiterals = [lMap[itr(l)] == model[l] for l in model if '\'' not in str(l)]
        # return the conjunction of all literals in this cube
    def cube(self):
        return And(*self.cubeLiterals)

    def __repr__(self):
```

```

        return str(self.t) + ": " + str(sorted(self.cubeLiterals, key=str))

class PDR(object):
    def __init__(self, literals, primes, init, trans, post):
        self.init = init
        self.trans = trans
        self.literals = literals
        self.lMap = {str(l):l for l in self.literals}
        self.post = post
        self.R = []
        self.primeMap = zip(literals, primes)

    def run(self):
        self.R = list()
        self.R.append(self.init)

        while(1==1):
            c = self.getBadCube()
            if(c != None):
                #print "Found bad cube:", c
                # we have a bad cube, which we will try to block
                # if the cube is blocked from the previous frame
                # we can block it from all previous frames
                trace = self.recBlockCube(c)
                if trace != None:
                    print ("Found trace ending in bad state:")
                    for f in trace:
                        print (f)
                    return False
            else: ## found no bad cube, add a new state on to R after checking for induction
                #print "Checking for induction"
                inv = self.checkForInduction()
                if inv != None:
                    print ("Found inductive invariant:", simplify(inv))
                    return True
                print ("Did not find invariant, adding frame", len(self.R))
                self.R.append(True)

# Check all images in R to see if one is inductive
def checkForInduction(self):
    for frame in self.R:
        s=Solver()
        s.add(self.trans)
        s.add(frame)
        s.add(Not(substitute(frame, self.primeMap)))
        if s.check() == unsat:
            return frame
    return None

#loosely based on the recBlockCube method from the berkely paper, without some of the optimizations
def recBlockCube(self, s0):
    Q = []
    Q.append(s0);
    while (len(Q) > 0):
        s = Q[-1]
        if (s.t == 0):
            # If a bad cube was not blocked all the way down to R[0]
            # we have found a counterexample and may extract the stack trace
            return Q

        # solve if cube s was blocked by the image of the frame before it
        z = self.solveRelative(s)

        if (z == None):
            # Cube 's' was blocked by image of predecessor:
            # block cube in all previous frames
            Q.pop() #remove cube s from Q
            for i in range(1, s.t+1):
                #if not self.isBlocked(s, i):
                self.R[i] = And(self.R[i], Not(s.cube()))
        else:
            # Cube 's' was not blocked by image of predecessor
            # it will stay on the stack, and z (the model which allowed transition to s) will be added on top
            Q.append(z)
    return None

#for tcube, check if cube is blocked by R[t-1] AND trans
def solveRelative(self, tcube):
    cubeprime = substitute(tcube.cube(), self.primeMap)
    s = Solver()
    s.add(self.R[tcube.t-1])
    s.add(self.trans)
    s.add(cubeprime)
    if(s.check() != unsat): #cube was not blocked, return new tcube containing the model
        model = s.model()
        return tCube(model, self.lMap, tcube.t-1)
    return None

# Using the top item in the trace, find a model of a bad state
# and return a tcube representing it
# or none if all bad states are blocked
def getBadCube(self):
    model = And(Not(self.post), self.R[-1])
    s = Solver()
    s.add(model)
    if(s.check() == sat):
        return tCube(s.model(), self.lMap, len(self.R) - 1)
    else:
        return None

# Is a cube ruled out given the state R[t]?
def isBlocked(self, tcube, t):

```

```

s = Solver()
s.add(And(self.R[t], tcube.cube()))
return s.check() == unsat

def isInitial(self, cube, initial):
s = Solver()
s.add (And(initial, cube))
return s.check() == sat

```

Vamos tentar provar a segurança do modelo da cifra através do algoritmo PDR. Vamos redefinir algumas funções como requerido pela implementação do PDR.

Poderemos então testar para valores arbitrários iniciais dos LFSR.

```

In [ ]: from z3 import *

LFSR_0 = BitVec('LFSR_0', 19)
LFSR_1 = BitVec('LFSR_1', 22)
LFSR_2 = BitVec('LFSR_2', 23)

LFSR_0_prime = BitVec('LFSR_0\'', 19)
LFSR_1_prime = BitVec('LFSR_1\'', 22)
LFSR_2_prime = BitVec('LFSR_2\'', 23)

variables = [LFSR_0, LFSR_1, LFSR_2]
primes = [LFSR_0_prime, LFSR_1_prime, LFSR_2_prime]

init = And(
    UGT(LFSR_0, BitVecVal(0, 19)),
    UGT(LFSR_1, BitVecVal(0, 22)),
    UGT(LFSR_2, BitVecVal(0, 23))
)

post = Or(
    LFSR_0 == BitVecVal(0, 19),
    LFSR_1 == BitVecVal(0, 22),
    LFSR_2 == BitVecVal(0, 23)
)

def trans():
    c0 = Extract(8, 8, LFSR_0)
    c1 = Extract(10, 10, LFSR_1)
    c2 = Extract(10, 10, LFSR_2)

    maj = (c0 & c1) | (c1 & c2) | (c2 & c0)

    updated_LFSR_0 = If(c0 == maj,
        Concat(Extract(17, 0, LFSR_0),
            Extract(18, 18, LFSR_0) ^ Extract(17, 17, LFSR_0) ^
            Extract(16, 16, LFSR_0) ^ Extract(13, 13, LFSR_0)),
        LFSR_0)

    updated_LFSR_1 = If(c1 == maj,
        Concat(Extract(20, 0, LFSR_1),
            Extract(21, 21, LFSR_1) ^ Extract(20, 20, LFSR_1)),
        LFSR_1)

    updated_LFSR_2 = If(c2 == maj,
        Concat(Extract(21, 0, LFSR_2),
            Extract(22, 22, LFSR_2) ^ Extract(21, 21, LFSR_2) ^
            Extract(20, 20, LFSR_2) ^ Extract(7, 7, LFSR_2)),
        LFSR_2)

    return And(
        LFSR_0_prime == updated_LFSR_0,
        LFSR_1_prime == updated_LFSR_1,
        LFSR_2_prime == updated_LFSR_2
    )

solver = PDR(variables, primes, init, trans, post)
solver.run()

```

Found trace ending in bad state:
0: [4194303 == LFSR_1, 524287 == LFSR_0, 8388607 == LFSR_2]

Out[]: False

Vamos agora testar para valores aleatórios iniciais dos LFSR.

```

In [ ]: from z3 import *
import random

LFSR_0 = BitVec('LFSR_0', 19)
LFSR_1 = BitVec('LFSR_1', 22)
LFSR_2 = BitVec('LFSR_2', 23)

LFSR_0_prime = BitVec('LFSR_0\'', 19)
LFSR_1_prime = BitVec('LFSR_1\'', 22)
LFSR_2_prime = BitVec('LFSR_2\'', 23)

variables = [LFSR_0, LFSR_1, LFSR_2]
primes = [LFSR_0_prime, LFSR_1_prime, LFSR_2_prime]

# valores aleatórios
random_LFSR_0 = BitVecVal(random.getrandbits(19), 19)
random_LFSR_1 = BitVecVal(random.getrandbits(22), 22)
random_LFSR_2 = BitVecVal(random.getrandbits(23), 23)

init = And(
    UGT(random_LFSR_0, BitVecVal(0, 19)),
    UGT(random_LFSR_1, BitVecVal(0, 22)),
    UGT(random_LFSR_2, BitVecVal(0, 23))
)

```

```

)

post = Or(
    LFSR_0 == BitVecVal(0, 19),
    LFSR_1 == BitVecVal(0, 22),
    LFSR_2 == BitVecVal(0, 23)
)

def trans():
    c0 = Extract(8, 8, LFSR_0)
    c1 = Extract(10, 10, LFSR_1)
    c2 = Extract(10, 10, LFSR_2)

    maj = (c0 & c1) | (c1 & c2) | (c2 & c0)

    updated_LFSR_0 = If(c0 == maj,
        Concat(Extract(17, 0, LFSR_0),
            Extract(18, 18, LFSR_0) ^ Extract(17, 17, LFSR_0) ^
            Extract(16, 16, LFSR_0) ^ Extract(13, 13, LFSR_0)),
        LFSR_0)

    updated_LFSR_1 = If(c1 == maj,
        Concat(Extract(20, 0, LFSR_1),
            Extract(21, 21, LFSR_1) ^ Extract(20, 20, LFSR_1)),
        LFSR_1)

    updated_LFSR_2 = If(c2 == maj,
        Concat(Extract(21, 0, LFSR_2),
            Extract(22, 22, LFSR_2) ^ Extract(21, 21, LFSR_2) ^
            Extract(20, 20, LFSR_2) ^ Extract(7, 7, LFSR_2)),
        LFSR_2)

    return And(
        LFSR_0_prime == updated_LFSR_0,
        LFSR_1_prime == updated_LFSR_1,
        LFSR_2_prime == updated_LFSR_2
    )

print(f"Valor aleatório inicial de LFSR_0: {random_LFSR_0}")
print(f"Valor aleatório inicial de LFSR_1: {random_LFSR_1}")
print(f"Valor aleatório inicial de LFSR_2: {random_LFSR_2}\n")

solver = PDR(variables, primes, init, trans, post)
solver.run()

```

```

Valor aleatório inicial de LFSR_0: 266240
Valor aleatório inicial de LFSR_1: 2726783
Valor aleatório inicial de LFSR_2: 3716037

```

```

Found trace ending in bad state:
0: [4194303 == LFSR_1, 524287 == LFSR_0, 8388607 == LFSR_2]

```

```
Out[ ]: False
```

Verificamos que, para qualquer valor aleatório que testamos, obtemos o mesmo mau traço, obtido também para o teste arbitrário.

Ao analisarmos estes valores, percebemos que:

- A representação binária de 4194303 é: 11111111111111111111.
- A representação binária de 524287 é: 11111111111111111111.
- A representação binária de 8388607 é: 1111111111111111111111.

Para quaisquer valores, o sistema chegará a este estado inseguro.

Portanto, a cifra A5/1 permite sempre a possibilidade de atingir um estado inseguro, independentemente da configuração inicial maior que zero.

Isto significa que a propriedade que está a ser verificada não é invariante para este sistema pois, para qualquer estado inicial aleatório, existe uma sequência de transições que pode levar pelo menos um LFSR a zero.